This is actually a better approach. Don't ask Copilot to build the whole system in one go. Give it **one well-scoped task**. Since you're currently building the **RAG pipeline**, the prompt should focus only on ingestion, indexing, retrieval, and testing—not the LLM generation.

Here's a prompt you can paste directly into GitHub Copilot Chat.

---

Edit

## Build the RAG Pipeline for a Threat Scenario Description Generator

You are an expert Python backend engineer, AI engineer, and LangChain developer.

I am building a Master's thesis project called **Threat Scenario Description Generator** for Industrial Control Systems (ICS).

At this stage, I only want to build the **RAG pipeline**. Do **not** implement the LLM generation yet. Focus only on knowledge ingestion, indexing, hybrid retrieval, and retrieval testing.

---

### Technology Stack

- Python 3.12
- LangChain
- ChromaDB
- Ollama
- nomic-embed-text embeddings
- BM25 Retriever
- FastAPI (later)
- Docker (later)

---

### Knowledge Sources

The knowledge base consists of:

- enterprise-attack.json (MITRE Enterprise ATT&CK)
- ics-attack.json (MITRE ICS ATT&CK)
- ICS Advisories (JSON or PDF converted to text)
- Future support for CVE/CWE

The design should allow adding more sources later without changing the retrieval logic.

---

### Goal

Implement a modular RAG system capable of:

1. Loading all knowledge sources.
2. Cleaning and normalizing documents.
3. Creating contextual chunks.
4. Generating embeddings using Ollama's `nomic-embed-text`.
5. Storing embeddings in ChromaDB.
6. Building a BM25 index.
7. Performing hybrid retrieval.
8. Combining retrieval results using Reciprocal Rank Fusion (RRF).
9. Returning the Top-K ranked chunks with metadata.

No LLM calls should be implemented yet.

---

## Project Structure

Create a clean project structure similar to:

```
rag/

    ingestion/
        loaders.py
        parser.py
        preprocessing.py

    chunking/
        contextual_chunker.py

    embeddings/
```

```
        embedding_service.py

    vectorstore/
        chroma_store.py

    retrieval/
        bm25_retriever.py
        vector_retriever.py
        hybrid_retriever.py
        rrf.py

    models/
        document.py

    utils/

    tests/
```

Use classes instead of large scripts.

---

## Document Model

Each chunk should contain:

- document_id
- chunk_id
- source
- title
- text
- metadata
- ATT&CK ID (if available)
- tactic
- technique
- platform
- references
- hash

Use dataclasses or Pydantic models.

---

## Contextual Chunking

Do NOT split documents using fixed-size chunking.

Instead, preserve semantic structure.

For MITRE ATT&CK JSON:

One chunk should represent:

- Technique
- Sub-technique
- Description
- Detection
- Mitigations
- Procedures
- Related Groups
- Related Software

For ICS advisories:

Chunk by advisory section rather than token count.

Chunks should preserve complete meaning.

Implement a reusable ContextualChunker class.

---

## Embeddings

Use LangChain with OllamaEmbeddings.

Model:

`nomic-embed-text`

Generate embeddings for every chunk.

Store:

- embeddings
- metadata
- original text

inside ChromaDB.

---

## BM25 Retrieval

Implement a separate BM25 retriever.

It should:

- tokenize chunks
- build an index
- return ranked chunks
- expose a retrieve(query, k) function

Do not mix BM25 logic with vector retrieval.

---

## Vector Retrieval

Implement a Chroma retriever using LangChain.

The retriever should return:

- chunk
- similarity score

---

## Hybrid Retrieval

Create a HybridRetriever class.

Workflow:

1. Receive query.
2. Retrieve top-k vector results.
3. Retrieve top-k BM25 results.
4. Merge duplicate chunks.
5. Apply Reciprocal Rank Fusion (RRF).
6. Return a ranked list.

The class should hide all retrieval complexity.

---

## Reciprocal Rank Fusion

Implement RRF separately.

Use

score = Σ (1 / (k + rank))

where k is configurable (default 60).

Allow multiple retrieval sources.

The implementation should be reusable.

---

## Retrieval Output

Return objects containing:

- chunk_id
- score
- source
- document_id
- metadata
- text

Sort by final RRF score.

---

## Testing

Write tests for:

- document loading
- contextual chunking
- embedding generation
- Chroma indexing
- BM25 retrieval
- vector retrieval
- hybrid retrieval
- RRF ranking

Provide a simple script that demonstrates:

```
Query

↓
```

```
Hybrid Retrieval

↓

Top 5 Retrieved Chunks

↓

Print metadata and scores
```

No LLM generation yet.

---

## Coding Standards

Use:

- SOLID principles
- Type hints
- Dataclasses/Pydantic
- Docstrings
- Modular architecture
- Dependency injection where appropriate

Avoid global variables.

Avoid monolithic scripts.

Write production-quality code that is easy to extend later with FastAPI and a Qwen2.5 generation module.

Generate one module at a time, ensuring each module is complete, documented, and testable before moving to the next.

This prompt is intentionally scoped to just the retrieval layer. Once this is working, your next Copilot prompt can focus on integrating **Qwen2.5-14B via Ollama**, building the deterministic prompt template, assembling retrieved context, and generating traceable threat scenarios. That staged approach is much more manageable than asking Copilot to build the entire thesis project in a single step.